

第2章 进程与线程

本章为 SmartSketch 项目自编的教学材料，用于参赛演示与抽取评测，不摘自任何教材或真实课程讲义。

2.1 本章导读

进程与线程是操作系统实现并发的核心概念。学习本章之前需要先掌握程序的执行过程与中断机制，因为进程切换依赖中断把控制权交还给操作系统。本章分为四部分：2.2 节介绍进程的概念与状态，2.3 节介绍进程控制与通信，2.4 节介绍线程，2.5 节讨论处理机调度。

2.2 进程

2.2.1 进程的定义

进程是程序在一个数据集合上的一次运行过程，是系统进行资源分配和调度的独立单位。程序是静态的指令集合，进程则是动态的执行实体：同一个程序运行两次，就产生两个不同的进程。进程具有动态性、并发性、独立性和异步性四个特征。

2.2.2 进程控制块

进程控制块（Process Control Block, PCB）是操作系统为每个进程维护的数据结构，是进程存在的唯一标志。PCB 中记录进程标识符、进程状态、程序计数器、寄存器内容、调度信息和打开的文件等。学习进程切换之前需要先掌握进程控制块，因为切换的本质就是保存和恢复 PCB 中的现场信息。

2.2.3 进程的状态

进程在生命周期中会在几种状态之间转换。就绪态是指进程已获得除处理机以外的全部资源，只等待被调度；运行态是指进程正在处理机上执行；阻塞态是指进程因等待某一事件（例如等待输入输出完成）而暂停执行。此外还有表示进程正在被创建的创建态和表示进程已经结束的终止态。

状态转换遵循固定规则：就绪态的进程被调度后进入运行态；运行态的进程时间片用完后回到就绪态；运行态的进程请求输入输出时进入阻塞态；等待的事件发生后，阻塞态的进程回到就绪态。阻塞态的进程不能直接进入运行态。学习进程调度之前需要先掌握进程的状态，因为调度只在就绪态的进程之间选择。

2.2.4 进程切换

进程切换是指处理机从一个进程转去运行另一个进程。切换时，操作系统先把当前进程的现场保存到它的 PCB 中，再从下一个进程的 PCB 中恢复现场。进程切换由中断或系统调用触发，需要在内核态下完成，因此切换本身会消耗处理机时间。

2.3 进程控制与通信

2.3.1 原语

原语是由若干条指令组成、执行期间不可被中断的程序段。操作系统用原语实现进程控制，例如创建原语、撤销原语、阻塞原语和唤醒原语。原语的不可分割性保证了进程状态和 PCB 的修改不会被并发操作破坏。

2.3.2 进程的创建与撤销

创建一个进程时，操作系统为它分配唯一的进程标识符和 PCB，分配所需的内存等资源，初始化 PCB，并把新进程插入就绪队列。由一个进程创建的进程称为子进程，创建者称为父进程，父子关系构成进程树。撤销进程时，操作系统回收它占有的全部资源，并把它的 PCB 从所在队列中移除。

2.3.3 进程通信

进程通信是指进程之间交换信息。由于进程拥有各自独立的地址空间，一个进程不能直接访问另一个进程的内存，因此需要操作系统提供通信机制。常见的方式有共享存储、消息传递和管道。

共享存储是指在通信的进程之间开辟一块双方都能访问的存储区，进程通过读写这块区域交换数据。消息传递是指进程通过发送原语和接收原语交换格式化的消息，消息可以直接发给对方，也可以经过称为信箱的中间实体。管道是连接一个读进程和一个写进程的共享文件，数据按先进先出的顺序流动，例如命令行中的“ls | grep txt”就用管道把前一个命令的输出交给后一个命令。

2.3.4 进程同步与互斥

进程同步是指多个相互合作的进程按照一定的先后次序执行。进程互斥是指当一个进程访问某种资源时，其他进程不能同时访问。一次只允许一个进程使用的资源称为临界资源，进程中访问临界资源的那段代码称为临界区。学习信号量之前需要先掌握进程同步与互斥，因为信号量正是为解决这两类问题而设计的。

信号量是一种用于同步和互斥的整型变量，只能通过 P 操作和 V 操作访问。P 操作将信号量减 1，若结果小于 0 则调用进程阻塞；V 操作将信号量加 1，若结果小于或等于 0 则唤醒一个等待的进程。例如，把初值为 1 的信号量用作互斥锁，进入临界区前执行 P 操作，离开时执行 V 操作，就能保证同一时刻只有一个进程在临界区内。

2.4 线程

2.4.1 线程的定义

线程是进程中的一个执行单元，是处理机调度的基本单位。一个进程可以包含多个线程，这些线程共享进程的地址空间、打开的文件等资源，但各自拥有独立的程序计数器、寄存器和栈。学习线程之前需要先掌握进程的定义，因为线程是在进程之内划分出的更小的执行单位。

引入线程的目的是减小程序并发执行时的时空开销。由于同一进程内的线程共享资源，线程切换不需要切换地址空间，因此比进程切换快得多。

2.4.2 进程与线程的比较

进程是资源分配的基本单位，线程是处理机调度的基本单位。同一进程中的线程切换不会引起进程切换；不同进程中的线程切换则会引起进程切换。进程之间相互独立，而同一进程的多个线程共享内存，因此线程之间通信更方便，但也更需要同步。

2.4.3 线程的实现方式

线程的实现分为用户级线程和内核级线程。用户级线程由应用程序中的线程库管理，内核不知道线程的存在，线程切换不需要进入内核态，速度快；但一个线程阻塞会导致整个进程阻塞。内核级线程由操作系统内核管理，一个线程阻塞时内核可以调度同一进程的其他线程，代价是线程切换需要进入内核态。

把用户级线程映射到内核级线程的方式称为多线程模型，包括多对一模型、一对一模型和多对多模型。例如，一对一模型为每个用户级线程创建一个对应的内核级线程，并发性好，但创建线程的开销较大。

2.5 处理机调度

2.5.1 调度的概念

处理机调度是指按照某种算法从就绪队列中选择一个进程，把处理机分配给它。调度分为高级调度、中级调度和低级调度：高级调度决定把哪些作业调入内存，中级调度决定把哪些暂时不能运行的进程换出到外存，低级调度决定把处理机分配给哪个就绪进程。

衡量调度算法的指标包括处理机利用率、系统吞吐量、周转时间、等待时间和响应时间。周转时间是作业从提交到完成所经历的时间；等待时间是进程在就绪队列中等待的时间总和。

2.5.2 常见调度算法

先来先服务调度算法按进程进入就绪队列的先后次序分配处理机，实现简单，但对短作业不利。短作业优先调度算法优先选择估计运行时间最短的进程，平均等待时间最短，但可能使长作业长期得不到调度，这种现象称为饥饿。

时间片轮转调度算法把处理机时间划分为固定长度的时间片，就绪进程按队列顺序轮流运行一个时间片，时间片用完而未结束的进程回到队尾。学习时间片轮转调度之前需要先掌握进程的状态，因为时间片用完会使进程从运行态回到就绪态。时间片过长会退化为先来先服务，过短则会使进程切换的开销过大。

优先级调度算法为每个进程设置优先级，总是选择优先级最高的就绪进程。多级反馈队列调度算法设置多个优先级不同的就绪队列，新进程先进入最高优先级队列，用完时间片仍未结束就降到下一级队列，它兼顾了短作业的响应速度和长作业的公平性。

2.6 本章小结

本章从进程的定义出发，介绍了进程控制块、进程状态及其转换和进程切换；讨论了原语、进程的创建与撤销、进程通信以及进程同步与互斥中的信号量；随后引入线程，比较了进程与线程，介绍了用户级线程和内核级线程；最后介绍了处理机调度的层次、评价指标和常见调度算法。下一章将讨论死锁，学习死锁之前需要先掌握本章的进程同步与互斥。